

花卉分类器 · 学习过程记录

本文记录从**零理解这个项目的过程**：概念是怎么一个个搞懂的、每个验证脚本回答了什么问题、犯过哪些错、每个错背后的规律是什么。

目的：留下“我是怎么学会的”的完整轨迹，而不只是交出一份结果。

一、学习路径总览

整个项目的知识主线：

图片数据 ≠ 表格数据

|

└ 1. 数据能不能用？

└ 2. 怎么分训练/验证？

└ 3. 样本不够怎么办？

└ 4. 尺寸不统一怎么办？

└ 5. 怎么喂给网络？

→ 坏图检测（两层校验）

→ 类内 8:2 切分 = 天然分层

→ 数据增强（只加在 train）

→ Resize + Normalize

→ Dataset（仓库）+ DataLoader（卡车）

|

└ 网络与训练

└ 6. 结构图 → 代码

└ 7. 输出的是概率吗？

└ 8. 怎么量“错多少”？

└ 9. 怎么更新参数？

└ 10. 怎么判断学没学到？

→ 尺寸公式 + 反推 padding

→ 是 logits；损失函数自带 log_softmax

→ CrossEntropyLoss（由任务形态决定）

→ 优化器 + 训练循环五步

→ train/val 对比 + loss 基线

学习方法（贯穿全程）：每个关键概念都写一个最小实验脚本跑出来，而不是只看解释。

这些脚本都在 docs/labs/，它们本身就是学习痕迹的证据。

二、时间线：概念是怎么一个个搞懂的

阶段 1 | 图片数据预处理的整体思路

起点问题：前面学的是表格数据（Titanic）的处理，图片数据有什么不同？

理解到的差异：

	表格数据	图片数据
一条样本	一行数字	一个 HWC 的三维数组
坏数据	缺失值、异常值	文件打不开、像素损坏
增强	基本没有	翻转 / 旋转 / 调色
统一化	归一化数值	Resize 到固定尺寸

图片数据的"范式"就是：**先保证每一条都能打开 → 再划分 → 再增强 → 再统一尺寸 → 最后装进 DataLoader。**

阶段 2 | 坏图检测为什么要两层 (clean.py)

从"能不能打开"到"判断得准不准":

```
with Image.open(path) as img:
    img.verify()          # 第一层：只查文件头/校验和，快
with Image.open(path) as img:  # verify() 后文件对象失效，必须重新打开
    img.load()            # 第二层：真正解码全部像素
```

- `verify()` 便宜但会漏（不解码像素，抓不到"头正常但数据截断"的图）；
- `load()` 彻底但慢；
- **两层组合才是完整判定。** 这个"快检 + 全检"的组合思路可以迁移到任何数据校验场景。

处置原则：原始数据只读，坏图移动而非删除，并生成 CSV 报告。
—— 数据是别人的，处理过程要可追溯、可回退。

阶段 3 | 划分：为什么"类内切 8:2"就天然分层

换了个问法：Titanic 里我用了 `stratify=y`，为什么这里不用？

答案：因为我是按类别循环、在每一类内部切。每类内都是 8:2，整体比例自动保持 8:2，不需要再单独分层。这正是分层抽样的意义 —— 防止某类被整体切进验证集。

阶段 4 | 管道必须「幂等」（一次真实的 bug）

现象：修正循环体 bug 后，其余四类都正好 20%，只有 tulip 是 24.4%。

定位过程：对比原始目录和输出目录的文件数，发现 tulip 盘上有 833 张，但原始只有 710 张 —— 凭空多了 123 个文件。

根因：`shutil.copy2` 只做"复制 + 覆盖同名"，永远不会删除多余文件。
上次跑剩的文件只要这次没被同名覆盖到，就一直躺在目录里，两次运行的结果取并集。

修法：`main()` 开头加 `reset_output()`，先清场再写盘。

学到的规律：幂等 —— 跑一次和跑十次，结果必须完全由输入决定，而不是由"上次留下了什么"决定。

阶段 5 | 缩进：Python 的缩进就是 C 的花括号（连续踩了三次）

三次踩坑，本质是同一件事：

现象	缩进位置	后果
代码掉到第 0 列	顶格	<code>for</code> 循环和函数被提前关闭 → <code>IndentationError</code>
代码掉到 4 格	出了循环体	只有最后一类有数据
变量被二次赋值	同一个变量	<code>train_loader = True</code> 覆盖了正确的 <code>DataLoader</code>

三条规律：

- 1. 一个块里所有代码行的缩进，必须与这个块的第一行完全相同。
动手前先问：“我这一行跟哪一行对齐？”
- 2. Python 没有块作用域（与 C 不同）：C 里 `for (int i...)` 的 `i` 出了循环就没了；
Python 里 `cls_dir / files` 循环结束后依然活着，保存着最后一轮的值。
- 3. 同一个变量赋值两次，只有最后一次算数，上一次的值直接被扔掉。

这三个错后来合成了同一句话：“我这一行/这个文件，属于哪个块/哪一次运行？”

阶段 6 | `shuffle` 到底改变了什么（本任务最值钱的一个判断）

最初的答案（错的）：“打乱不影响正确率，只是改变了参数的位置。”

修正：

- 打乱改变的是样本被取出的顺序，不是“参数的位置”（评估时参数根本不更新）；
- “打乱不影响正确率”只在评估时成立——因为参数冻结，正确率 = 答对数 ÷ 总数，与顺序天然无关；
- 训练时打乱是必须的。

用真实数据验证这个必要性：

`ImageFolder` 按文件夹名排序装载，所以不打乱时数据是按类别一块块排着的。

实测 2425 张、batch=32：

配置	前 12 个 batch 的组成	单一类别 batch 占比
<code>shuffle=False</code>	batch 1~12 全是 <code>daisy × 32</code>	96.1% (73/76)
<code>shuffle=True</code>	<code>[11,2,9,2,8]</code> 、 <code>[4,2,10,4,12]</code> ...（每车五类齐全）	0%

后果：不打乱 → 每个 batch 的梯度是单一类别的平均方向，
相当于让模型“先学完雏菊、再学蒲公英...”一段段地掰，还会被最后见到的类别带偏。
打乱之后每个 batch 覆盖五类，模型每一轮都在“同时看五种花”。

最终口诀：打乱顺序不改变「评估」，但改变「学习」。分界线是参数有没有在更新。

阶段 7 | 结构图 → PyTorch 代码（只一条公式）

```
输出边长 = [(输入边长 + 2×padding - kernel) / stride] + 1
```

图上写明的照抄，没写的（padding）用公式反推。反推三步：

- 1. 抄 —— 抄下 In / K / S 和目标 Out ；
- 2. 试 —— 把 P = 0,1,2,3... 逐个代入，看哪个能让 Out 对上；
- 3. 选 —— 含向下取整，可能有多个 P 同时命中，取最小的那个。

反解式（一步求最小 P）：

```
P_min = ceil( (stride × (Out - 1) - In + kernel) / 2 )
```

conv1 的"差 1"陷阱：padding=0 代入得 [(224-11)/4]+1 = 54，但结构图要 55。

这不是图错了，而是出题人默认第一层补了边（即 AlexNet 论文里 224/227 输入尺寸争议的来源）。

反解得 P = [(4×54 - 224 + 11)/2] = [1.5] = 2。

（补充：P=2 和 P=3 都能得到 55，按惯例取最小的 2。）

逐层验证结果（9 步全部与结构图一致）：

```
224 → conv1(55) → pool1(27) → conv2(27) → pool2(13)
      → conv3(13) → conv4(13) → conv5(13) → pool3(6) → flatten(9216)
```

阶段 8 | 损失函数：只由「任务形态」决定

核心认知：模型输出的是 logits（原始分数，可正可负、加起来不为 1），它自己不知道好坏；损失函数把它和正确答案对比成一个数字，这个数字就是梯度的源头。

选哪把尺子不看偏好，看任务形态：

任务形态	用什么
5 选 1 分类	nn.CrossEntropyLoss()
二分类	nn.BCEWithLogitsLoss()
回归	nn.MSELoss()

两个必须记住的细节：

- 1. CrossEntropyLoss 内部自带 log_softmax，所以 forward 末尾绝不能再加 Softmax；
- 2. labels 必须是整数下标（long 型），不能是 one-hot、不能是 float。
实测：传 float 会报 expected target dtype to be Long or Byte, but got Float。

阶段 9 | 优化器与学习率的「配对」

最深刻的一次教训（详见 训练过程记录.md 实验 0）：

模型 15 轮完全不学习，loss 卡在 $\ln(5)=1.6094$ （= 瞎猜）。
根因不是"SGD 不行"，而是 SGD 配了 $lr=1e-4$ —— 这个量级是给 Adam 的，差 100 倍。

优化器	怎么决定步长	常用 lr
Adam	按梯度历史自动缩放	$1e-4 \sim 3e-3$
SGD	完全按给定的 lr 走	$1e-2 \sim 1e-1$

学到的调参原则：先检查超参数之间的适配关系，再去比较谁更优。
(本次：配对错误造成约 38 个点的差距；优化器之间本身只差约 2 个点。)

三、四个验证脚本各证明了什么

这些脚本是"把机制跑出来"的记录，都在 docs/labs/。

3.1 size_lab.py —— 尺寸公式与 padding 反推

- 验证正向公式：给定 In/K/S/P 算 Out；
- 验证反推：给定目标 Out 反求 P；
- 亲手撞上 conv1 的"差 1"陷阱，并用反解式求出 $P=2$ 。

3.2 data_loader_lab.py —— DataLoader 四个参数

用假数据（10 张假图）跑了四组实验：

实验	结论
A 仓库与卡车	$\text{len}(ds)=10$ ，一次取一条； $\text{batch_size}=4 \rightarrow 3$ 个 batch，第一车形状 $(4,3,4,4)$ 。首维 4 = batch 大小，这就是 $[B,C,H,W]$ 里 B 的来源
B shuffle	<code>False</code> $\rightarrow [0,1,2,0,1,2\dots]$ （原顺序）； <code>True</code> $\rightarrow$ 真的乱了。每轮 epoch 开头都会重新打乱
C drop_last	<code>False</code> $\rightarrow 3$ 个 batch（尾巴 2 张保留）； <code>True</code> $\rightarrow 2$ 个（尾巴扔掉）
D num_workers	0 和 2 取出的东西完全一样，区别只在"谁来搬货"。Windows 下 >0 必须写 <code>if __name__ == "__main__":</code>

3.3 train_lab.py —— 训练循环的机制

最值钱的是实验 B：

```

after 1st backward      w.grad = 2.0
2nd backward, no zeroing w.grad = 5.0  ← 2 + 3, 上一步的还在！
zeroed, then 3rd backward w.grad = 3.0  ← 只有这一步的

```

`loss.backward()` 执行的是 `grad +=`，不是 `grad =`。

这就是循环第一步必须是 `zero_grad()` 的全部原因。

另外还验证了：`optimizer.step()` 就是 $w \leftarrow w - lr \times grad$ （实测 $5 \rightarrow 4 \rightarrow 3.2 \rightarrow 2.56$ ）、

`model.train()` / `eval()` 对 Dropout 的开关作用（不开 `train()`，Dropout 不生效）。

3.4 lr_lab.py —— 定位“学不动”的根因

固定数据与初始权重，只换优化器与 lr，这样“学不动”的责任就精确落到 lr 头上，而不用怀疑模型或数据。读表方法：

total drop 的样子	说明
≈ 0	参数几乎没动 —— 对应失败配置
明显变大 (>0.05)	步长配得上这个网络
NaN / 数值乱跳	lr 太大

说明：最终没有依赖这个脚本下结论 —— 而是直接用两组完整的 15 轮真实训练（实验 0 vs 实验 1/2）作为证据，说服力更强。

3.5 compare_history.py —— 从 json 重画对照曲线

从 `docs/logs/history_*.json` 重画，想比几组就比几组，永远不用重训。

准确率图里那条灰点线是“五分类瞎猜 = 20%”基线，一眼看出学没学到。

四、犯过的错与修正（按性质分类）

4.1 「报错行 ≠ 案发行」家族（4 个成员）

报错	它指的位置	真凶在哪
<code>IndentationError: unexpected indent</code>	第 75 行	第 50 行掉回第 0 列
<code>'bool' object has no attribute 'dataset'</code>	第 60 行（使用处）	第 48 行（覆盖赋值）
<code>SyntaxError ... Perhaps you forgot a comma?</code>	第 34 行（下一个元素）	上一行漏了逗号
曲线图 Loss 子图空白	图本身	改代码比训练进程启动晚了 3 分钟

统一判据：见到这类"位置和病因对不上"的报错，先往上游找一遍，再动手改。

先问一句："这条信息是谁、什么时候、哪一次产生的？"

4.2 「Python 与 C 的差异」家族

主题	C 的直觉	Python 的真相
缩进	只是排版， {} 才是语法	缩进本身就是花括号， 写错就是语法错误
作用域	for 的变量出了循环就没了	没有块作用域， 循环变量在循环后仍存活， 保留最后一轮的值
搜索路径	gcc -I.. ma.c 控制	sys.path[0] = 被运行脚本所在目录； 被 import 的模块用不上自己补的路径
运行方式	gcc *.c -o app 整体编译	没有"编译整个项目"， 必须点名到一个具体文件

4.3 「环境与运行方式」家族

现象	真相
ModuleNotFoundError: No module named 'config'	不是逻辑错 —— transforms.py 一直被 import， 自己单独跑时暴露了"没补 sys.path"
can't find '__main__' module	命令给了目录 (python src\)， Python 把它当包去找 __main__.py
报错里出现 uv\python\...\python.exe	用的是别的解释器， 那里没装 torch —— 要写全路径 D:\ML\venv_gpu\Scripts\python.exe

4.4 「API 形态」类（不是不懂原理，是不知道那一行长什么样）

写成了	应该写成	病因
self.conv1 = (3,96,kernel_size=11,stride=4)	nn.Conv2d(3,96,kernel_size=11,stride=4,padding=2)	括号前没有名字时它是元组，不是调用；元组里不能写 key=value
nn.Maxpool2d(...)	nn.MaxPool2d(...)	Pool 的 P 要大写
self.pool = nn.Conv2d(kernel_size=3,stride=2)	nn.MaxPool2d(kernel_size=3,stride=2)	池化不是卷积； Conv2d 还必须给输入/输出通道数
train_loader = True	DataLoader(...)	占位符把正确的赋值覆盖了

核心认知： `self.conv1` 里装的是"一台造好的机器（层对象）"，不是"图纸上的数字"。

`nn.Conv2d(...)` 是"按参数造机器"，`self.conv1(x)` 才是"让机器干活"。

给自己的教训：遇到新 API 时，先弄清它的"形态"（长什么样），再填"参数"（填什么值）——这两件事是独立的，我以前常把它们混在一起。

4.5 环境小坑

- 数据"凭空变多" → 管道不幂等（见阶段 4）；
- 训练耗时异常（epoch 11 用了 684s、epoch 14 用了 1993s）→ 电脑休眠，`time.time()` 把挂起时间也算进去了，**不影响结果**。

五、这个项目之后，我还想搞清楚的

问题	为什么想搞清楚
为什么要用 <code>Dropout</code> ，它到底在做什么？	本项目的全连接层占了 94.5% 的参数， <code>Dropout</code> 是防过拟合的关键，但只理解了"随机丢弃"这一层
感受野怎么算？为什么小核堆叠更好？	结论记住了（"两个 $3 \times 3 =$ 一个 5×5 ，但参数更少、非线性更多"），推导还没自己走过一遍
迁移学习为什么能大幅提升？	选做部分 V 要用预训练权重，想先弄懂"预训练学到了什么"
学习率衰减（scheduler）怎么用？	SGD 长跑时前快后稳的常用手段
<code>BatchNorm</code> 起什么作用？	AlexNet 原版没有，但现代网络几乎都有

六、我的学习方式小结

回顾整个过程，真正让我搞懂的不是"看解释"，而是这三件事：

1. **每个概念写一个最小实验跑出来** —— 用假数据、不碰真数据，把机制跑一遍亲眼看到；
2. **用真实数据做定量验证** —— 比如"不打乱时 96.1% 的 batch 只有一类"，一个数字比一段描述有说服力得多；
3. **踩坑之后沉淀出可复用的判据** —— 比如"报错行 $\neq$ 案发行"这一行属于哪个块"，这些判据后来反复救了我。

保留 `docs/labs/` 下这些脚本、保留 `# TODO:` 痕迹、保留每一版失败的训练记录 —— 都是为了让"我是怎么想的、怎么错的、怎么改的"能被看见。